Optimal Execution under Market Impact

Reinforcement Learning Execution Agent

Adrián Vázquez

May 26, 2026

Abstract

This section extends the optimal execution framework toward adaptive decision-making through Reinforcement Learning.

Building on the previously developed market state representation, execution environment, and benchmark execution policies, we implement a Deep Q-Network (DQN) agent designed to learn execution decisions directly from sequential interaction with the market environment.

The agent observes both execution-related variables, such as remaining inventory and remaining execution horizon, and market-related variables, including liquidity conditions, rolling volatility, momentum, relative volume, and participation-related features.

Unlike deterministic benchmark schedules such as TWAP, VWAP-like execution, and Volume-Aware Almgren–Chriss, the Reinforcement Learning framework allows execution intensity to dynamically adapt according to the observed market state.

The objective of this section is to evaluate whether a learning-based execution agent can discover coherent state-dependent execution behavior capable of balancing:

- inventory liquidation,
- execution cost,
- and participation impact

under realistic intraday market conditions.

The learned policy is evaluated against classical benchmark execution strategies through cumulative execution cost, participation dynamics, inventory trajectories, and execution stability analysis.

Contents

1 Reinforcement Learning Execution Agent **3**

1.1 Objective . . . 3
1.2 State Representation . . . 3
1.3 Execution Environment . . . 4
1.4 Benchmark Policies . . . 4
1.5 Benchmark Baseline Results . . . 5
1.6 Deep Q-Network (DQN) . . . 5
1.7 Replay Buffer . . . 6
1.8 Epsilon-Greedy Exploration . . . 6
1.9 Target Network . . . 6
1.10 DQN Training Process . . . 7
1.11 Learned Policy Evaluation . . . 7
1.12 Learned Policy Results . . . 8
1.13 Learned Policy Behavior Analysis . . . 8
1.13.1 Inventory Remaining by Policy . . . 9
1.13.2 Cumulative Execution Cost by Policy . . . 10
1.13.3 Participation Rate by Policy . . . 10
1.13.4 DQN Learned Execution Schedule . . . 11

2 Conclusions **11**

3 Future Work **12**

1 Reinforcement Learning Execution Agent

1.1 Objective

The previous sections developed the execution framework progressively:

- Classical execution schedules (TWAP, VWAP, Almgren–Chriss)
- Stochastic execution simulation
- Real intraday execution analysis
- Volume-aware execution environments

At this stage, the objective is no longer to define static execution schedules, but instead to learn adaptive execution behavior directly from market interaction.

The goal of the Reinforcement Learning framework is to answer:

- Can an agent learn how aggressively to execute according to market conditions?
- Can execution decisions adapt dynamically to liquidity and inventory exposure?
- Can adaptive execution outperform static benchmark schedules?

1.2 State Representation

The RL agent observes both:

- execution state,
- and market state.

The execution state includes:

- remaining inventory,
- remaining execution horizon.

The market state includes:

- liquidity conditions,
- rolling volatility,
- momentum,
- relative volume,
- and participation-related features.

The final state representation is defined as:

$$s_t = [x_t, T - t, \text{market features}]$$

This allows the agent to condition execution decisions on both inventory exposure and observed market conditions.

1.3 Execution Environment

The Reinforcement Learning environment developed in the previous section is reused as the execution simulator.

The environment models:

- sequential trade execution,
- participation-based market impact,
- inventory dynamics,
- and execution rewards.

At each timestep:

1. The agent observes the current market state.
2. An execution action is selected.
3. Shares are executed in the market.
4. Execution cost and participation impact are computed.
5. The environment transitions to the next state.

The reward structure penalizes:

- execution cost,
- excessive participation,
- and remaining inventory risk.

1.4 Benchmark Policies

Before training the RL agent, several benchmark execution policies are used as reference strategies:

- Random execution
- TWAP
- VWAP-like execution

- Volume-Aware Almgren–Chriss (VA-AC)

These policies provide deterministic and liquidity-sensitive baselines against which the learned RL execution policy can be evaluated.

1.5 Benchmark Baseline Results

Before evaluating the learned Reinforcement Learning policy, the benchmark execution strategies are first executed under the same market environment.

These results establish the baseline execution performance level that the DQN agent must compete against.

Table 1: Benchmark Execution Policy Results

Policy	Cum. Cost	Shares Traded	Final Inv.	Avg. Part.	Max Part.	Steps
Random	-18114.97	100000	0	0.0101	0.0285	40
TWAP	-22810.39	100000	0	0.0048	0.0100	78
VWAP-like	-44048.57	100000	0	0.0033	0.0036	78
VA-AC	-33344.17	100000	0	0.0039	0.0057	78

The results show that liquidity-aware execution policies outperform naive execution schedules under realistic market conditions.

In particular, the VWAP-like strategy achieved the lowest cumulative execution cost among the benchmark policies, suggesting that incorporating observed market liquidity substantially improves execution quality.

1.6 Deep Q-Network (DQN)

A Deep Q-Network (DQN) architecture is implemented to learn adaptive execution policies directly from market state observations.

The neural network maps:

- execution state,
- liquidity conditions,
- volatility,
- and inventory exposure

into discrete execution actions representing execution intensity.

The DQN framework consists of:

- Policy Network
- Target Network

- Replay Buffer
- Epsilon-Greedy exploration

1.7 Replay Buffer

The Replay Buffer stores past execution experiences of the form:

$$(s_t, a_t, r_t, s_{t+1}, done)$$

where:

- s_t = current state
- a_t = selected execution action
- r_t = reward received
- s_{t+1} = next observed state

Instead of learning sequentially from highly correlated observations, the agent samples random mini-batches from memory.

This stabilizes training and improves learning efficiency.

1.8 Epsilon-Greedy Exploration

The DQN agent selects actions using an epsilon-greedy policy.

With probability:

$$\epsilon$$

the agent explores by selecting random execution actions.

With probability:

$$1 - \epsilon$$

the agent exploits the learned Q-network by selecting the action with the highest estimated Q-value.

This balances:

- exploration of new execution behaviors,
- and exploitation of learned execution policies.

1.9 Target Network

A separate target network is used to stabilize training dynamics.

Instead of updating target Q-values using a constantly changing network, the target network is periodically synchronized with the policy network.

This reduces instability during Q-learning optimization and improves convergence behavior.

1.10 DQN Training Process

The DQN agent learns execution behavior through sequential interaction with the execution environment.

At each timestep, the agent:

- observes the current execution and market state,
- selects an execution action,
- executes shares in the market,
- receives an execution reward,
- and observes the resulting next state.

The resulting transition:

$$(s_t, a_t, r_t, s_{t+1}, done)$$

is stored in the Replay Buffer.

Mini-batches of past experiences are then sampled to iteratively update the Q-network by minimizing the difference between:

- current Q-value estimates,
- and target Q-values computed from future rewards.

Through repeated interaction with the market environment, the agent progressively learns execution policies balancing:

- execution cost,
- participation impact,
- and inventory liquidation risk.

Unlike deterministic benchmark schedules, the learned policy dynamically adapts execution intensity according to the observed market state.

1.11 Learned Policy Evaluation

After training, the DQN agent is evaluated under the same execution environment used by the benchmark strategies.

During evaluation, exploration is disabled by setting:

$$\epsilon = 0$$

allowing the agent to execute trades exclusively according to the learned policy.

The learned execution policy is then compared against:

- Random execution
- TWAP
- VWAP-like execution
- Volume-Aware Almgren–Chriss (VA-AC)

The evaluation focuses on:

- cumulative execution cost,
- inventory liquidation,
- participation dynamics,
- and execution stability.

1.12 Learned Policy Results

Table 2: DQN Policy Evaluation Results

Policy	Cum. Cost	Shares Traded	Final Inv.	Avg. Part.	Max Part.	Steps
DQN	-57806.34	100000	0	0.0047	0.0253	76
Random	-18114.97	100000	0	0.0101	0.0285	40
TWAP	-22810.39	100000	0	0.0048	0.0100	78
VA-AC	-33344.17	100000	0	0.0039	0.0057	78
VWAP-like	-44048.57	100000	0	0.0033	0.0036	78

The DQN agent achieved the lowest cumulative execution cost among all evaluated execution policies.

Despite being trained through exploration-based interaction, the learned execution trajectory remained stable and coherent, successfully liquidating the full inventory while maintaining controlled participation behavior.

Compared to deterministic benchmark schedules, the RL agent learned adaptive state-dependent execution behavior, dynamically adjusting execution intensity according to observed market conditions.

1.13 Learned Policy Behavior Analysis

After training, the DQN agent is evaluated against benchmark execution policies:

- Random execution
- TWAP
- VWAP-like execution

- Volume-Aware Almgren–Chriss (VA-AC)

The objective is not only to minimize cumulative execution cost, but also to analyze how the agent behaves under realistic market conditions.

Key evaluation dimensions include:

- cumulative execution cost,
- inventory liquidation dynamics,
- participation rate control,
- execution aggressiveness,
- and execution trajectory stability.

Unlike static benchmark policies, the DQN agent dynamically adapts execution intensity according to the observed market state.

1.13.1 Inventory Remaining by Policy

Figure 1: Inventory Remaining by Policy

The DQN agent learned an aggressive inventory liquidation behavior, prioritizing early execution and rapid inventory reduction over liquidity-sensitive pacing.

Compared to VWAP-like and Volume-Aware Almgren–Chriss policies, the learned strategy reduced inventory substantially faster during the first half of the trading horizon, suggesting that the agent implicitly prioritized inventory risk minimization and future uncertainty reduction.

Despite its aggressiveness, the learned execution trajectory remained smooth and stable, indicating that the agent discovered a coherent execution policy rather than random liquidation behavior.

1.13.2 Cumulative Execution Cost by Policy

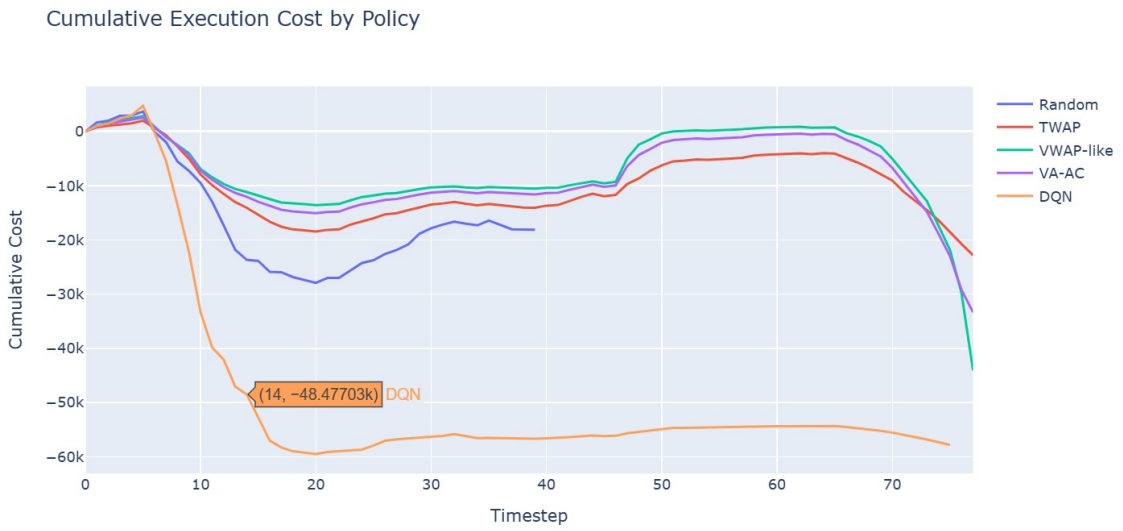

Figure 2: Cumulative Execution Cost by Policy

The DQN agent converged toward a front-loaded execution strategy, rapidly reducing inventory exposure during the early trading horizon.

This behavior produced the lowest cumulative execution cost among all benchmark policies, suggesting that the learned policy strongly prioritized inventory risk reduction and early uncertainty minimization.

The resulting execution trajectory reflects an implicit trade-off between aggressive early liquidation and passive liquidity-sensitive execution.

1.13.3 Participation Rate by Policy

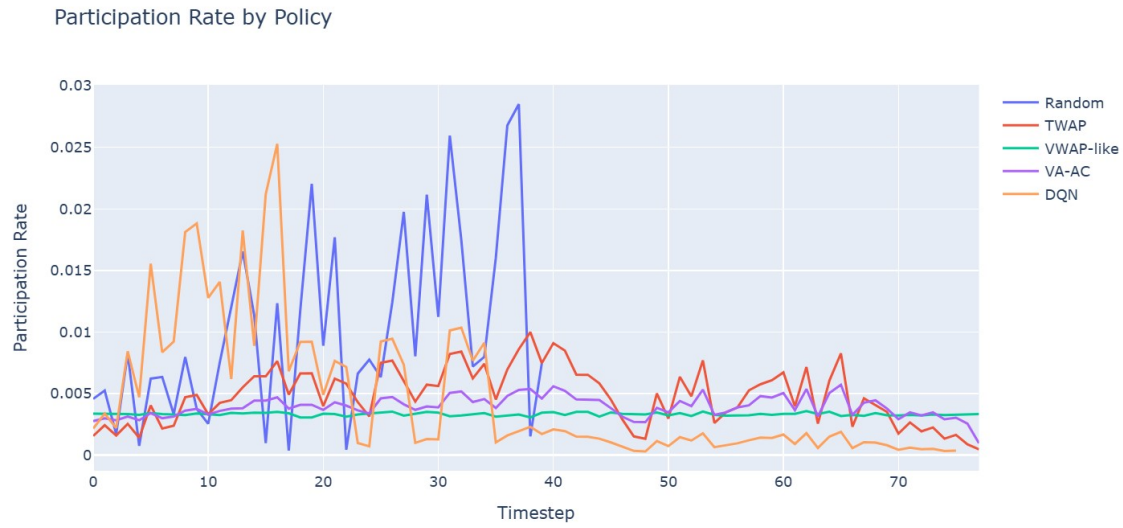

Figure 3: Participation Rate by Policy

The learned DQN policy exhibited temporally adaptive participation behavior.

The agent executed aggressively during the early trading horizon, reaching higher participation rates while inventory exposure remained elevated, and gradually transitioned toward lower participation as inventory was reduced.

This suggests that the agent learned a dynamic execution profile balancing early inventory reduction with later impact moderation.

1.13.4 DQN Learned Execution Schedule

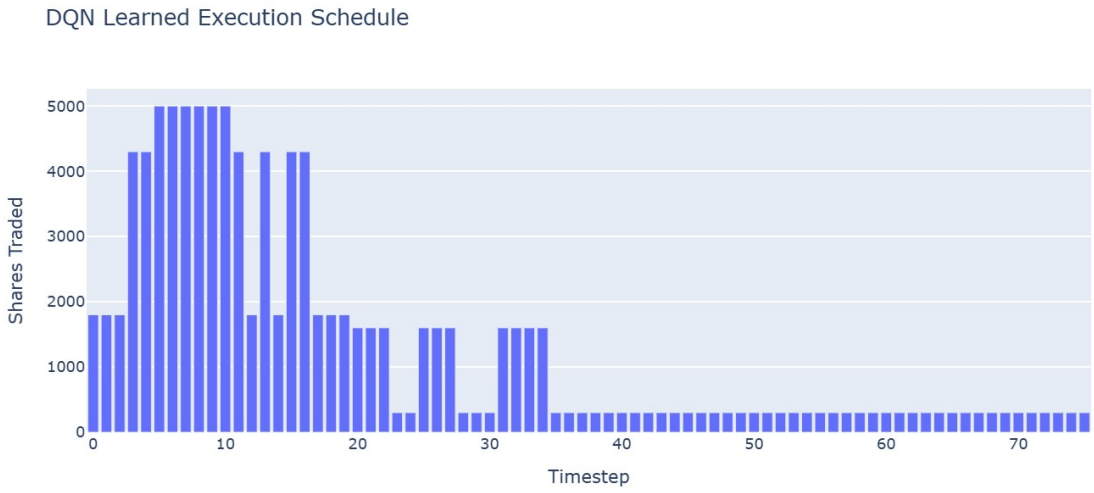

Figure 4: DQN Learned Execution Schedule

The learned execution schedule exhibited a strongly front-loaded structure, characterized by large execution blocks during the early trading horizon followed by progressively smaller trades as inventory exposure decreased.

Rather than converging toward a static execution schedule, the DQN agent learned a state-dependent execution profile that dynamically adjusted trading intensity throughout the execution horizon.

This behavior suggests that the agent implicitly identified different execution regimes, balancing early inventory liquidation with later participation moderation.

2 Conclusions

The Reinforcement Learning execution agent successfully learned a coherent execution policy under realistic intraday market conditions.

Key findings include:

- The DQN agent learned a strongly front-loaded execution strategy, prioritizing early inventory liquidation during the first half of the trading horizon.

- The learned policy dynamically adjusted participation intensity over time, exhibiting aggressive execution while inventory exposure remained elevated and transitioning toward lower participation as inventory decreased.
- Compared to benchmark policies such as TWAP, VWAP-like execution, and Volume-Aware Almgren–Chriss, the DQN agent achieved the lowest cumulative execution cost during the evaluated trading session.
- Despite being trained through exploration-based reinforcement learning, the learned execution trajectory remained smooth and stable, avoiding erratic liquidation behavior.
- Inventory and participation dynamics suggest that the agent implicitly learned the trade-off between inventory risk reduction and market impact moderation.

However, an important limitation remains.

The learned execution policy was evaluated on the same market environment used during training, meaning that the observed performance may partially reflect environment-specific adaptation rather than true generalizable execution intelligence.

In particular, the learned policy still appears biased toward aggressive early liquidation rather than fully liquidity-sensitive execution timing.

As a result, an important open question emerges:

> Can the learned RL execution policy generalize to unseen market conditions and different intraday liquidity regimes?

This motivates the next stage of the project: evaluating the DQN agent under out-of-sample market environments.

3 Future Work

Future work includes:

- out-of-sample evaluation across multiple trading days,
- robustness analysis under different volatility regimes,
- and generalization testing under changing liquidity conditions.

References

[1] Almgren, R., & Chriss, N. (2001). *Optimal Execution of Portfolio Transactions.*

[2] Cartea, Á., Jaimungal, S., & Penalva, J. (2015). *Algorithmic and High-Frequency Trading.*

[3] Ning, B., Treichler, D., & Chen, S. (2021). *Double Deep Q-Learning for Optimal Execution.* Applied Mathematical Finance.